

Large Language Models with Humans in the Loop

From Training to Generation, Conversations and Interactions

François Yvon

ISIR — CNRS and Sorbonne Université



Master Androïde

Part I

Language modeling: the foundations

Modeling syntax, probabilistically

A ubiquitous problem: comparing sentences

A: *I do not like to teach in the morning*

B: *I do not like teaching in the morning*

A better than B or B better than A?

Optimal probabilistic decision rule

A better than B $\Leftrightarrow P(A) \geq P(B)$

$P(\text{sentence})$ is a language model

Modeling syntax, probabilistically

A ubiquitous problem: comparing sentences

A: *I do not like to teach in the morning*

B: *I do not like teaching in the morning*

A better than B or B better than A?

Ranking sentences in natural language generation tasks

<i>les enfants ont</i>	<i>danser</i> <i>dansés</i> <i>dansé</i> <i>dansée</i>	<i>sur la plage</i>
------------------------	---	---------------------

Useful for speech transcription, OCR correction, spelling / grammar checking, , text generation, machine translation ...

Optimal probabilistic decision rule

Modeling syntax, probabilistically

A ubiquitous problem: comparing sentences

A: *I do not like to teach in the morning*

B: *I do not like teaching in the morning*

A better than B or B better than A?

Optimal probabilistic decision rule

A better than B $\Leftrightarrow P(A) \geq P(B)$

$P(\text{sentence})$ is a language model

Modeling syntax, probabilistically

A ubiquitous problem: comparing sentences

A: *I do not like to teach in the morning*

B: *I do not like teaching in the morning*

A better than B or B better than A?

Optimal probabilistic decision rule

A better than B $\Leftrightarrow P(A) \geq P(B)$

$P(\text{sentence})$ is a **language model**

A Markov model for natural languages

The simplest sequence model: n -gram



$$\begin{aligned} P(w_1 \dots w_L) &= \prod_{i=1}^L P(w_i | w_1 \dots w_{i-1}) \\ &= \prod_{i=1}^L P(w_i | w_{i-n+1} \dots w_{i-1}) \end{aligned}$$

Language Modeling - the historical perspective

The task of the **language Model is to estimate the probabilities** $P(w_t | w_1 \dots w_{t-1})$. It is of course absurd to think that the speaker's choice of the t^{th} words actually depends on the entire history $h_t = w_1 \dots w_{t-1}$ of his previous speech.

Assuming w_t are discrete symbols “(...) **the art of language modeling consists of determining the appropriate classification Φ and the estimation of the probabilities $P(w_t | \Phi(h_{t-1}))$** ”

A Markov model for natural languages

The simplest sequence model: n -gram



$$\begin{aligned} P(w_1 \dots w_L) &= \prod_{i=1}^L P(w_i | w_1 \dots w_{i-1}) \\ &= \prod_{i=1}^L P(w_i | w_{i-n+1} \dots w_{i-1}) \end{aligned}$$

- **Markov assumption:** given recent history $h = w_{i-n+1} \dots w_{i-1}$, remote words are unimportant.
- linguistically naïve, computationally efficient.
- very old model (Markov, Shannon)

A Markov model for natural languages

The simplest sequence model: n -gram



$$\begin{aligned}
 P(w_1 \dots w_L) &= \prod_{i=1}^L P(w_i | w_1 \dots w_{i-1}) \\
 &= \prod_{i=1}^L P(w_i | w_{i-n+1} \dots w_{i-1})
 \end{aligned}$$

n -gram text generation with ancestral sampling

$$w_1 \sim \text{Unif}(W_1); w_2 \sim P(W_2 | w_1); w_3 \sim P(W_3 | w_2 w_1) \dots$$

Length L is not modelled: make sure to know when to stop sampling

A Markov model for natural languages

The simplest sequence model: n -gram



$$\begin{aligned} P(w_1 \dots w_L) &= \prod_{i=1}^L P(w_i | w_1 \dots w_{i-1}) \\ &= \prod_{i=1}^L P(w_i | w_{i-n+1} \dots w_{i-1}) \end{aligned}$$

Process more than words: letters, phones, morphs, clauses, speech turns, etc

An effective model of sequences without hierarchical structures

n-gram models are so simple, yet so tricky to estimate

Zipf's curse strikes back

Parameter estimation from running texts

Maximum likelihood estimates (with 2-word histories: **trigrams**)

$$P(w|uv) = \frac{c(uvw)}{\sum_{w'} c(uvw')} \quad (1)$$

$c()$ is the **count** function, $h = uv$ is the **history**

The art of language modeling

- with 100,000 words, $100,000^3$ 3-gram counts, most of them 0
- build **history classes** ($uv \rightarrow h(uw)$) to keep models small
- building history classes ? the science of **count smoothing** [1992-2012]

n-gram models are so simple, yet so tricky to estimate

Zipf's curse strikes back

Parameter estimation from running texts

Maximum likelihood estimates (with 2-word histories: **trigrams**)

$$P(w|uv) = \frac{c(uvw)}{\sum_{w'} c(uvw')} \quad (1)$$

$c()$ is the **count** function, $h = uv$ is the **history**

The art of language modeling

- with 100,000 words, $100,000^3$ 3-gram counts, **most of them 0**
- build **history classes** ($uv \rightarrow h(uw)$) to keep models small
- building history classes ? the science of **count smoothing** [1992-2012]

The art of smoothing

Smoothing is some **black magic**: estimate the probability of unseen events

“+1”-smoothing (Laplace):

$$P_{+1}(w|h) = \frac{c(hw)+1}{|\mathcal{V}| + c(h)}$$

Variant (add $+\alpha$, with α a parameter). Alternatives

- Linear interpolation

- Clustering: building word / history classes

- Discounting techniques (Katz back-off, Good-Turing back-off, Knesser-Ney smoothing)

- Non-parametric Bayesian models

A side effect of smoothing

Before smoothing, many word sequences have a null probability;
After smoothing, **all word sequences have a positive probability**.

The art of smoothing

Smoothing is some **black magic**: estimate the probability of unseen events

“+1”-smoothing (Laplace):

$$P_{+1}(w|h) = \frac{c(hw)+1}{|\mathcal{V}| + c(h)}$$

Variant (add $+\alpha$, with α a parameter). Alternatives

- Linear interpolation

- Clustering: building word / history classes

- Discounting techniques (Katz back-off, Good-Turing back-off, Knesser-Ney smoothing)

- Non-parametric Bayesian models

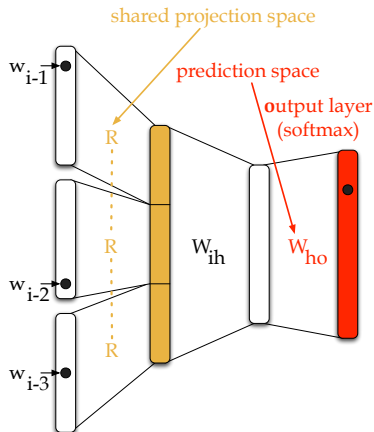
A side effect of smoothing

Before smoothing, many word sequences have a null probability;

After smoothing, **all word sequences have a positive probability**.

Feed-forward language models [Bengio et al., 2003]

Continuous space n -gram models



$$\mathbf{i} = [w_{i-1}^T R, w_{i-2}^T R, w_{i-3}^T R]$$

$$\mathbf{h} = \mathbf{i}^T W_{ih} + \mathbf{b}_{ih}$$

$$\mathbf{o} = \tanh(\mathbf{h})^T W_{ho} + \mathbf{b}_{ho}$$

$$P(w_i | w_{i-3}, w_{i-2}, w_{i-1}) = \frac{\exp \mathbf{o}[w_i]}{\sum_w \exp \mathbf{o}[w]}$$

What neural language modeling does:

- ① encodes history as $\phi(w_{i-3}, w_{i-2}, w_{i-1})$
- ② compares $\phi(w_{i-3}, w_{i-2}, w_{i-1})$ and $R'(w_i)$

Feed-forward language models [Bengio et al., 2003]

Continuous space n -gram models

Training FFLMs - maximize log-likelihood / cross-entropy

$$\begin{aligned}\theta^* = [\mathbf{R}, \mathbf{W}_{ih}, \mathbf{b}_i, \mathbf{W}_{ho}, \mathbf{b}_o] &= \operatorname{argmax}_{\theta} \sum_t \log \frac{\exp \mathbf{o}[w_t]}{\sum_w \exp \mathbf{o}[w]} \\ &= \operatorname{argmax}_{\theta} \sum_t \mathbf{o}[w_t] - \log \left(\sum_w \exp \mathbf{o}[w] \right)\end{aligned}$$

- $\text{softmax}(\mathbf{x}) = \left[\frac{\exp \mathbf{x}[i]}{\sum_k \exp \mathbf{x}[k]} \right]$ computes **dense distributions**
- **embeddings** $\mathbf{R}$ are learned
- **computationally demanding** (softmax layer)
- superior to discrete (n-gram) LMs [Schwenk, 2007, Le et al., 2012]

Feed-forward language models [Bengio et al., 2003]

Continuous space n -gram models

<i>word (freq.)</i>	<i>model</i>	<i>5 nearest neighbors</i>
<i>is</i> 900,350	standard 1 vector init. (*)	was are were been remains was are be were been
<i>conducted</i> 18,388	standard 1 vector init. (*)	undertaken launched \$270,900 Mufamadi 6.44-km-long pursued conducts commissioned initiated executed
<i>Cambodian</i> 2,381	standard 1 vector init. (*)	Shyorongi \$3,192,700 Zairian depreciations teachers' Danish Latvian Estonian Belarussian Bangladeshi
<i>automatically</i> 1,528	standard 1 vector init. (*)	MSSD Sarvodaya \$676,603,059 Kissana 2,652,627 routinely occasionally invariably inadvertently seldom
<i>Tosevski</i> 34	standard 1 vector init. (*)	\$12.3 Action,3 Kassouma 3536 Applique Shafei Garvalov Dostiev Bourloyannis-Vrailas Grandi
<i>October-12</i> 8	standard 1 vector init. (*)	39,572 anti-Hutu \$12,852,200 non-contracting Party's March-26 April-11 October-1 June-30 August4
<i>3727th</i> 1	standard 1 vector init. (*)	Raqu Tatsei Ayatallah Mesyats Langlois 4160th 3651st 3487th 3378th 3558th

(*) 1 vector init: share parameters R and W_{ho} during init.

[Examples from [Le et al., 2010]]

FFLMs induce similarities between histories and between words

The infamous < unk > word

“Closed world” assumptions

- The support of LM : a fixed vocab $\mathcal{V}$. Sentences with unknowns have 0 probability.
- The support of LM : a fixed vocab $\mathcal{V} \cup \{ < \text{unk} > \}$.
Estimation: all words $\notin \mathcal{V}$ are **unked** [makes < unk > very likely].
- Variant: consider classes of < unk > (proper names, numbers, etc).

“Open” world models with subword units: morphemes, char ngrams, bytes, pixels

- morph-based LM: require morphological analysis, < unk > still possible
- letters: no more unknown words - **unknown symbols instead ?**
- a mixture of words and letters

Caveats

- Shorter units require longer histories [estimation problems]
- And imply longer sentences [computational problems]

The infamous < unk > word

“Closed world” assumptions

- The support of LM : a fixed vocab $\mathcal{V}$. Sentences with unknowns have 0 probability.
- The support of LM : a fixed vocab $\mathcal{V} \cup \{ < \text{unk} > \}$.
Estimation: all words $\notin \mathcal{V}$ are **unked** [makes < unk > very likely].
- Variant: consider classes of < unk > (proper names, numbers, etc).

“Open” world models with subword units: morphemes, char ngrams, bytes, pixels

- morph-based LM: require morphological analysis, < unk > still possible
- letters: no more unknown words - **unknown symbols instead ?**
- a mixture of words and letters

Caveats

- Shorter units require longer histories [estimation problems]
- And imply longer sentences [computational problems]

The infamous < unk > nown word

“Closed world” assumptions

- The support of LM : a fixed vocab $\mathcal{V}$. Sentences with unknowns have 0 probability.
- The support of LM : a fixed vocab $\mathcal{V} \cup \{ < unk > \}$.
Estimation: all words $\notin \mathcal{V}$ are **unked** [makes < unk > very likely].
- Variant: consider classes of < unk > (proper names, numbers, etc).

“Open” world models with subword units: morphemes, char ngrams, bytes, pixels

- morph-based LM: require morphological analysis, < unk > still possible
- letters: no more unknown words - **unknown symbols instead** ?
- a mixture of words and letters

Caveats

- Shorter units require longer histories [**estimation problems**]
- And imply longer sentences [**computational problems**]

Subword Units in LLMs: BPEs, wordpieces, etc

Byte pair encoding: N deterministic merge operations

- ① Make symbol map (greedy)
Repeat till done: merge most frequent bigram into new compound symbol
- ② Encode (greedy)
split each word into compound symbols

Subword Units in LLMs: BPEs, wordpieces, etc

Byte pair encoding: N deterministic merge operations

- 1 Make symbol map (greedy)
Repeat till done: merge most frequent bigram into new compound symbol
- 2 Encode (greedy)
split each word into compound symbols

Example from [Sennrich et al., 2016]

$L = \{ \text{lower, lowest, newer, wider, wide} \}$

e	r#	3	[er#]		[lo]	w	2	[low]
l	o	2	[lo]		w	i	2	[wi]

Segmentations: [low]+[er#], [low]+e+s+[t#], n+e+w+[er#], [wid]+[er#],
[wid]+[e#]

🔗 <https://github.com/rsennrich/subword-nmt>

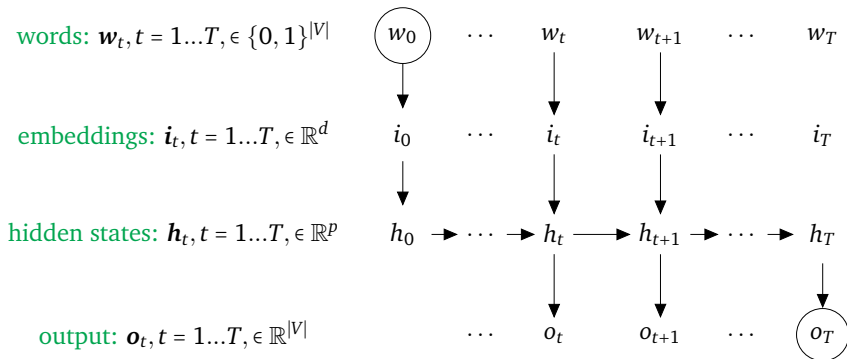
Subword Units in LLMs: BPEs, wordpieces, etc

Byte pair encoding: N deterministic merge operations

- 1 Make symbol map (greedy)
Repeat till done: merge most frequent bigram into new compound symbol
 - 2 Encode (greedy)
split each word into compound symbols
- better solutions: SentencePiece (unigram model), wordPiece
 - also **regularization** through sampling
 - handles **multilingual vocabularies** [use with care]
 - segmentation is a **latent variable**
 $P(w|h)$ requires summing over segmentations [Cao and Rimell, 2021]

Recurrent Neural Networks as LMs

From finite to infinite contexts - a rediscovery of [Mikolov et al., 2010]



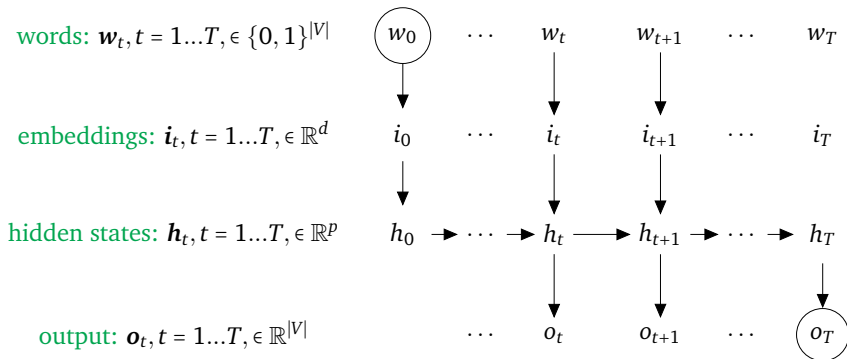
$$\mathbf{i}_t = [\mathbf{w}_t^T \mathbf{R}]$$

$$\mathbf{h}_t = \tanh(\mathbf{i}_t^T \mathbf{W}_{ih} + \mathbf{h}_{t-1}^T \mathbf{W}_{hh} + \mathbf{b}_{ih})$$

$$\mathbf{o} = \mathbf{h}^T \mathbf{W}_{ho} + \mathbf{b}_{ho}$$

Recurrent Neural Networks as LMs

From finite to infinite contexts - a rediscovery of [Mikolov et al., 2010]

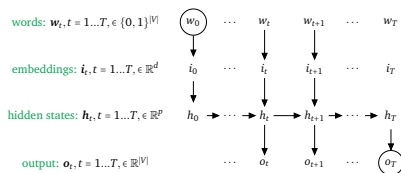


$$\theta^* = \underset{\theta}{\operatorname{argmax}} \sum_t \log \mathbf{o}[w_t] - \log \left(\sum_w \exp \mathbf{o}[w] \right)$$

$$P(w_{t+1} = k | w_{\leq t}; \theta_{LM}) = \operatorname{softmax}(\mathbf{o}_t = W^{ho} \mathbf{h}_t + \mathbf{b}^o)[k]$$

Recurrent Neural Networks as LMs

From finite to infinite contexts - a rediscovery of [Mikolov et al., 2010]



- train with word prediction objective and cross-entropy loss
- more complex **cells** ($(w_t, h_t) \rightarrow h_{t+1}$): GRUs, LSTMs
- same issues as FFLMs with softmax, same solutions apply
- **stack** several hidden layers $h_t^k = f(h_{t-1}^k, h_t^{k-1})$: biRNNs, etc.
- h_T : compact, effective **sentence representation** for $w_1 \dots w_T$
- **backwards processing** computes $\bar{h}_{-1}$
- $[h_T, \bar{h}_{-1}]$ a **better representation**: text classification, etc.
- $[h_t, \bar{h}_t]$ represents word w_t **and its context**

RNNs as “pure” encoders

An approach to sentence classification

$\mathbf{h}_T = \text{RNN}(w_1 \dots w_T)$ encodes a **variable-length sentence** in a **fixed-length vector**.

Decision rule for **TC tasks**, mapping sentences to classes (sentiment / opinion mining, stance detection, entailment, etc): $w_1 \dots w_T \rightarrow y$

$$P(y = 1 \mid w_1 \dots w_T) = \sigma(\mathbf{W}^T \mathbf{h}_T + b)$$

$$\theta^* = \operatorname{argmax}_i \sum \log P(y^{(i)} \mid w_1^{(i)} \dots w_{T(i)}^{(i)})$$

Improved classification results with backward encoding, multiple layers etc.

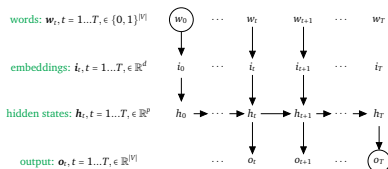
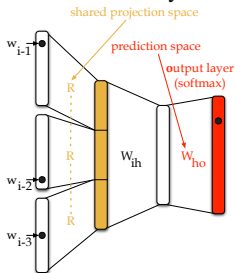
Also works with **pairs of sentences** (textual entailment, discourse modeling):

$$\mathbf{h}_T^w = \text{RNN}(w_1 \dots w_T), \mathbf{h}_S^v = \text{RNN}(v_1 \dots v_S)$$

$$h^{v,w} = \mathbf{h}_T^w \oplus \mathbf{h}_S^v \text{ or } h^{v,w} = [\mathbf{h}_T^w, \mathbf{h}_S^v] \text{ or } \dots$$

Self-attention: cooking complex histories

FFLMs / RNNs predict next word based on a **continuous representation of the history** with trained similarity



- FFLMs use a **fixed-size history**
- RNNs use a **pre-defined structure**: all words matter, **recent words matter more than distant words**.

New architecture: SAN-LM (aka **Transformers**) [Vaswani et al., 2017] trained to **learn which history words matter**.

Self-attention: cooking complex histories

Heads: parameterized computing modules

Query parameters: $\mathbf{H}_q \in \mathbb{R}^l \times \mathbb{R}^d$

Key parameters: $\mathbf{H}_k \in \mathbb{R}^l \times \mathbb{R}^d$

Value parameters: $\mathbf{H}_v \in \mathbb{R}^o \times \mathbb{R}^d$

Computing attention with heads

Heads linearly transform **matrices** $\mathbf{I} \in \mathbb{R}^d \times \mathbb{R}^T$ into **matrices** $\mathbf{O}$ in $\mathbb{R}^o \times \mathbb{R}^T$.

transform input matrix for words: $\mathbf{J} = \mathbf{H}_q \times \mathbf{I} \in \mathbb{R}^l \times \mathbb{R}^T$

transform input matrix for contexts: $\mathbf{K} = \mathbf{H}_k \times \mathbf{I} \in \mathbb{R}^l \times \mathbb{R}^T$

transform input matrix for outputs: $\mathbf{L} = \mathbf{H}_v \times \mathbf{I} \in \mathbb{R}^o \times \mathbb{R}^T$

compute similarities words/context: $\mathbf{D} = \mathbf{J} \times \mathbf{K}^T \in \mathbb{R}^T \times \mathbb{R}^T$

compute linear weights: $\tilde{\mathbf{D}} = \text{softmax}\left(\frac{\mathbf{D}}{\sqrt{d}}\right) \in [0, 1]^T \times [0, 1]^T$ columnwise

linear combination of cols: $\mathbf{O} = \tilde{\mathbf{D}} \times \mathbf{L} \in \mathbb{R}^T \times \mathbb{R}^o$

Self-attention: cooking complex histories

Using several Heads [in parallel]: MultiHeads

- one Head : $\mathbf{I}(d \times T) \rightarrow \mathbf{O}(o \times T)$
- k Heads : $\mathbf{I}(d \times T) \rightarrow [\mathbf{O}_1, \dots, \mathbf{O}_k](ko \times T)$

Using multiple layers of MultiHeads

- compute with k Heads : $\mathbf{I}(d \times T) \rightarrow \mathbf{O} = [\mathbf{O}_1 \dots \mathbf{O}_k](ko \times T)$
- enable **residual** (direct) connections $\mathbf{O}' = \mathbf{O} + \mathbf{I}$
- pass $\mathbf{O}'$ through a “linear” layer $\mathbf{O}'' = \mathbf{O}' + \mathbf{W}' \times \text{RELU}(\mathbf{W}\mathbf{O})$, with $\mathbf{O}'' \in \mathbb{R}^{(d \times T)}$
- make layers and sublayers comparable through **layer normalization** (subtract mean, divide by stddev)
- stack multiple layers $\mathbf{I}_1 \rightarrow \mathbf{I}_2 \rightarrow \mathbf{I}_3 \rightarrow \mathbf{I}_4 \dots$

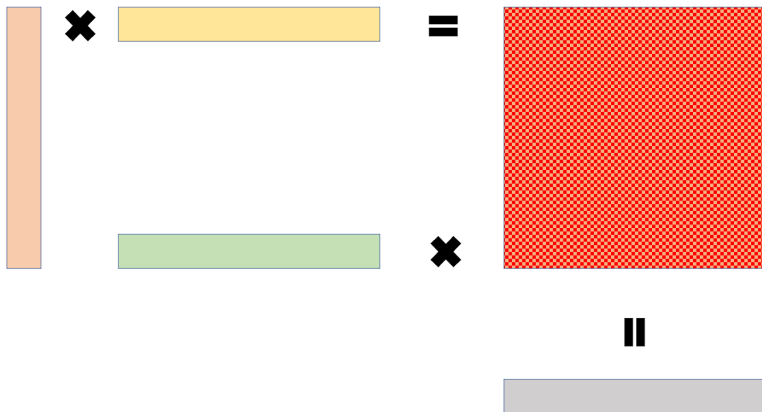
Typical values: 8 Heads of output dimension $o = 64$, 6 – 12 layers of heads of dimension 512.

Self-attention: cooking complex histories



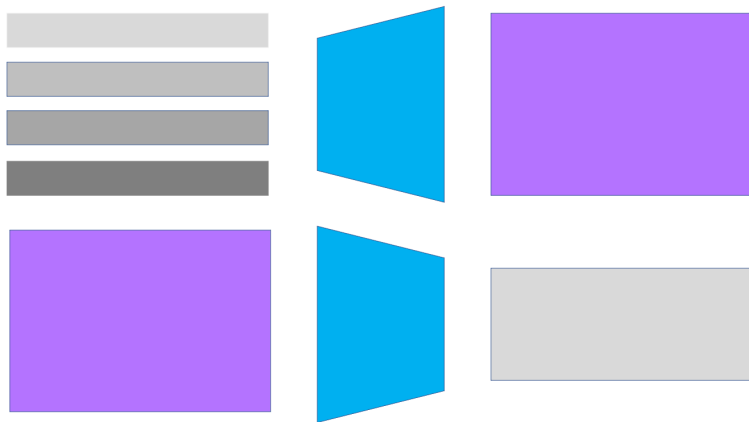
Computes J, K, L for 4 Heads

Self-attention: cooking complex histories



Computes $\tilde{D}$ and K

Self-attention: cooking complex histories



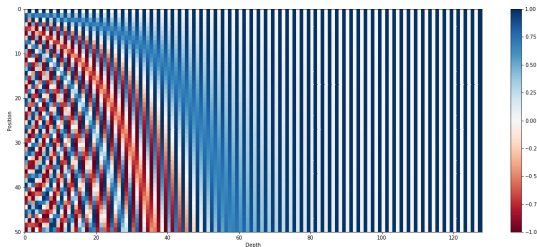
Computes output

Self-attention: cooking complex histories

The initial encoding layer

Input = $w_1 \dots w_T$, columns in $\mathbf{I}_1$ combine **word embeddings** and **positional encodings**.

$$\begin{cases} \mathbf{P}[2i, t] = \sin(t/10000^{2i/d}) \\ \mathbf{P}[2i + 1, t] = \cos(t/10000^{2i/d}) \end{cases} \in \mathbf{P} \in \mathbb{R}^d \times \mathbb{R}^T$$



Also trainable PEs, relative PEs [Raffel et al., 2020], Alibi PEs [Press et al., 2022] etc.

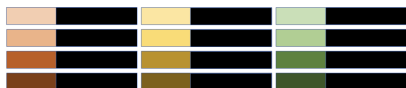
Self-attention: cooking complex histories

Masking & Causality in Language Modeling

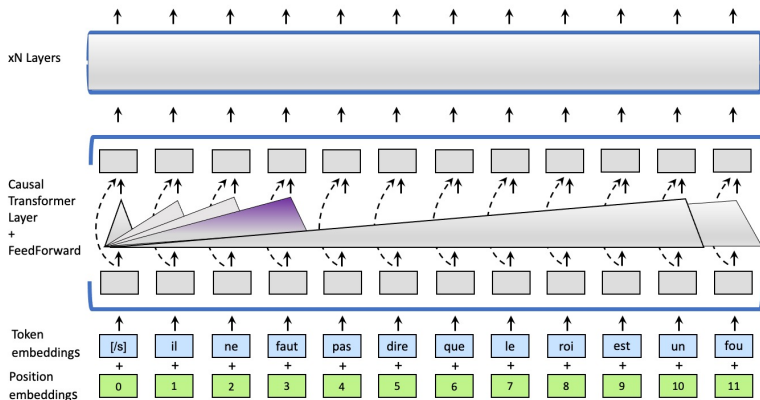
The past should not depend on the future.

Causal / Masked LMs **mask** ([_]) future time steps

context	predicts
$\langle s \rangle w_1 \dots w_t \square \square \dots$	w_{t+1}
$\langle s \rangle w_1 \dots w_{t+1} \square \square \dots$	w_{t+2}



Self-attention: cooking complex histories



Incremental computation: fully causal transformer

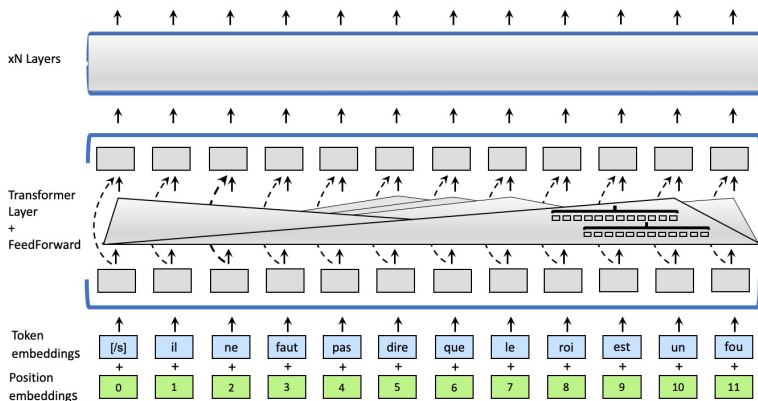
graphics, explanations <https://jalammar.github.io/illustrated-gpt2/>

Self-attention: cooking complex histories

Output layer and prediction (MLM)

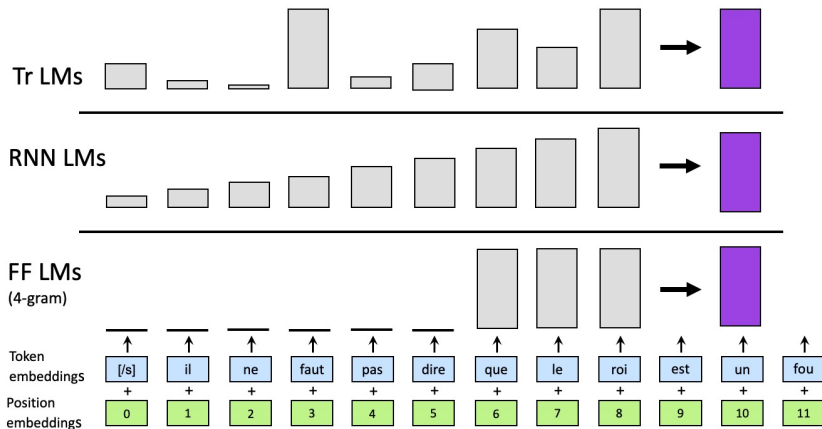
- pack hidden states $[\mathbf{I}_K[t], \mathbf{I}_{K-1}[t], \mathbf{I}_{K-2}[t], \dots]$ into $\sum_u \theta_u \mathbf{I}_u[t]$
- project into $\mathbb{R}^{|\mathcal{V}|}$ to get logits: $g(\mathbf{W}_o(\sum_u \theta_u \mathbf{I}_u[t]) + \mathbf{b}_o)$
- use softmax to predict next word w_{t+1}
- compute loss $\ell_{ce} = -\log P(w^* | w_{<t}) = \text{KL}(\mathbb{I}(w^*) \parallel P(W | w_{<t}))$
- back-prop gradient

Self-attention: cooking complex histories



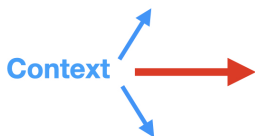
Non causal transformers are contextual representation learners

Self-attention: cooking complex histories



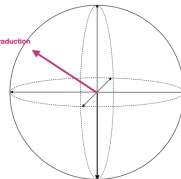
Neural Predictions - from Vectors to Probabilities

La langue de l'Europe, c'est la traduction



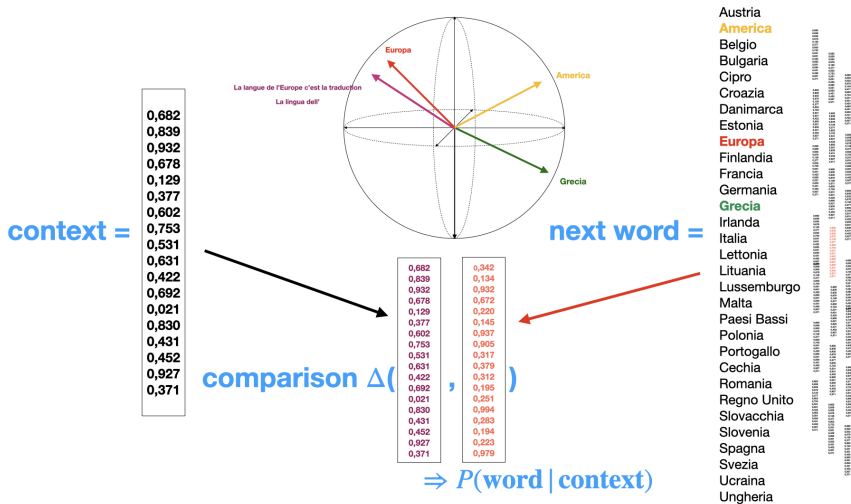
0.662
0.269
0.932
0.678
0.129
0.277
0.602
0.753
0.531
0.631
0.422
0.692
0.021
0.630
0.431
0.452
0.827
0.271

La langue de l'Europe c'est la traduction
La lingua dell'Europa

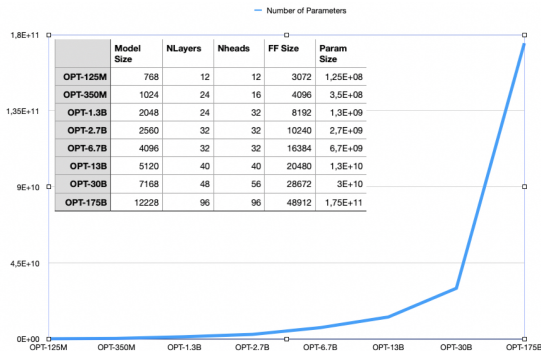


La lingua dell' Europa

Neural Predictions - from Vectors to Probabilities



Scaling up language models to “foundational” models



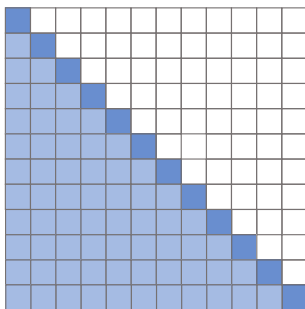
Dimensions of very large language models (OPT from Meta)

The **GPT-style family**: GPT1-4 (OpenAI), Megatron (Nvidia), Gopher / Chinchilla (DeepMind), HyperClova (NaverLabs), GPT-J, Pythia (EuletherAI), OPT (Meta), Bloom (BigScience), Llama[1-3] (Meta), Palm, Gemma (Google), Mistral, Qwen, DeepSeek ...

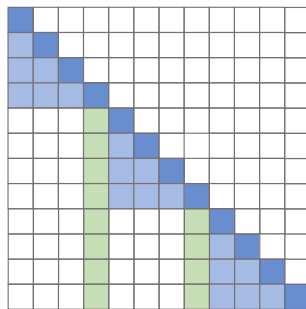
Scaling up language models to “foundational” models

Algorithmic issues [Tay et al., 2022]:

- extended context length T : costs $O(T^2)$ + positional embeddings + gradients storage
- increased depth - costs gradients storage



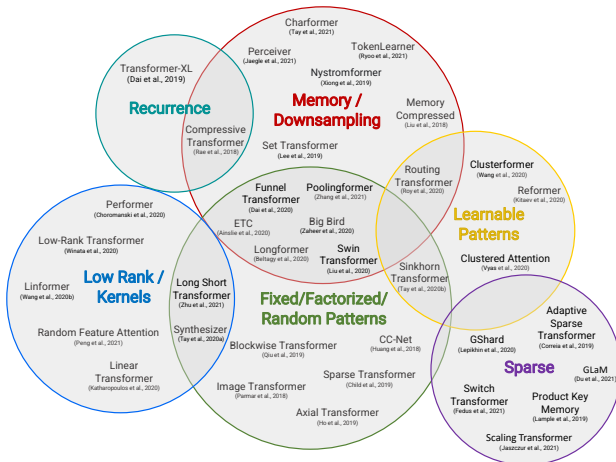
full attention



local attention + sentinels

- also: random or hierarchical sparse attention masks, parameter sharing, approximate dot product computation, better positional encodings, etc

Scaling up language models to “foundational” models



A bestiary of efficient transformers [Tay et al., 2022]

Large Language Models are *very* powerful

Natural language processing tasks, such as question answering, machine translation, reading comprehension and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision (...) [Radford et al., 2019b]

- ✓ scoring model for disambiguation tasks (as any language model)
- ✓ continuous representations of sentences and paragraphs: towards **transfert learning**
- ✓ simple and effective generation mechanism
$$\text{Gen}(w_1 \dots w_T) = \operatorname{argmax}_{w_{t+1} \dots w_T} P(w_{t+1} \dots w_T | w_1 \dots w_t)$$
- ✓ use generation as multitask processing with prompts / instructions

Large Language Models are *very* powerful

Natural language processing tasks, such as question answering, machine translation, reading comprehension and summarization, are typically approached with supervised learning on task-specific datasets. We demonstrate that language models begin to learn these tasks without any explicit supervision (...) [Radford et al., 2019b]

Prompt (ANLI)	The Gold Coast Hotel & Casino is a hotel and casino located in Paradise, Nevada. This locals' casino is owned and operated by Boyd Gaming. The Gold Coast is located one mile west of the Las Vegas Strip on West Flamingo Road. It is located across the street from the Palms Casino Resort and the Rio All Suite Hotel and Casino. Question: The Gold Coast is a budget-friendly casino. True, False, or Neither?
Answer (OK)	Neither
Answer (KO)	True
Answer (KO)	False
Prompt (PIQA)	How to apply sealant to wood ?
Answer (OK)	Using a brush, brush on sealant onto wood until it is fully saturated with the sealant.
Answer (KO)	Using a brush, drip on sealant onto wood until it is fully saturated with the sealant.
Prompt (COPA)	My body cast a shadow over the grass because
Answer (OK)	the sun was rising.
Answer (KO)	the grass was cut.

☞ examples from Radford et al. [2019b]

Also: summarization / compression, machine translation (!), arithmetic (!!)

Part II

Training and Using LLMs

Pre-Training Foundation Models

LLM Pretraining - maximize log-likelihood a.k.a cross-entropy

Observing text $W = w_1 \dots w_T$:

$$\begin{aligned}
 \theta^* &= \operatorname{argmax}_{\theta} \ell(\theta) = \log P(w_1 \dots w_T | \theta) \\
 &= \operatorname{argmax}_{\theta} \log \prod_t P(w_t | w_{<t}; \theta) && \text{Chain rule} \\
 &= \operatorname{argmax}_{\theta} \sum_t \log \frac{\exp \operatorname{logit}(w_t, w_{<t}; \theta)}{\sum_{w' \in \mathcal{V}} \exp \operatorname{logit}(w', w_{<t}; \theta)} \\
 &= \operatorname{argmax}_{\theta} \sum_t \operatorname{logit}(w_t, w_{<t}; \theta) - \log \left(\sum_w \exp \operatorname{logit}(w, w_{<t}; \theta) \right)
 \end{aligned}$$

Facts:

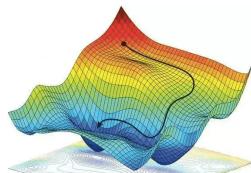
- The log-likelihood ℓ is a **differentiable function** of θ , θ is a very very large vector, structured in layers.
- Training = optimization with **stochastic gradient descent**, using mini batches containing a fraction of W . Differentiation uses the chain rule - is computed by autodiff tools.
- Pretraining does not require any human labeling effort.

Pre-Training Foundation Models

Stochastic Gradient Rule

$$\theta_{t+1} \leftarrow \theta_t - \eta \bar{\nabla} \ell(\theta),$$

with $\bar{\nabla} \ell(\theta)$ an **estimate** of the gradient

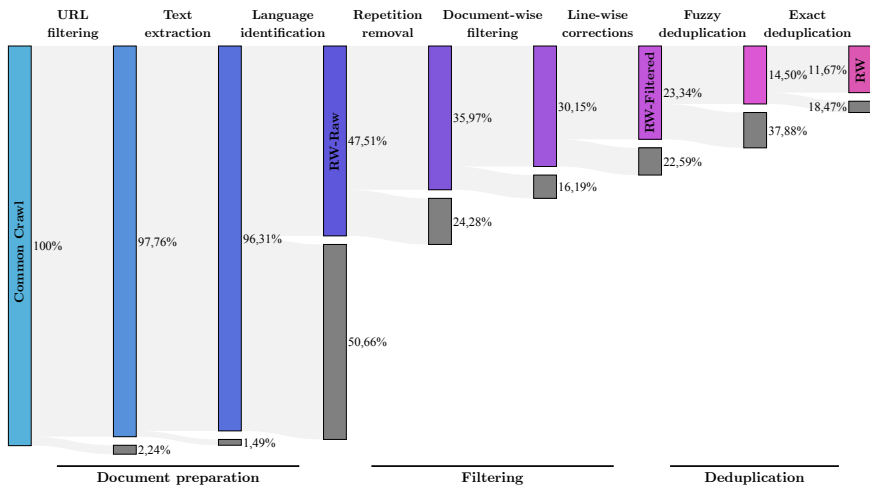


Controlling training:

- with more efficient optimization techniques and their **meta parameter values** (batch size, time varying gradient steps, early stopping, etc)
- with better loss functions (e.g. *label smoothing*) and regularization techniques (e.g., **drop out**)
- with the right amount of data samples per parameter - **scaling laws**

Preparing Pretraining Data : Design Choices

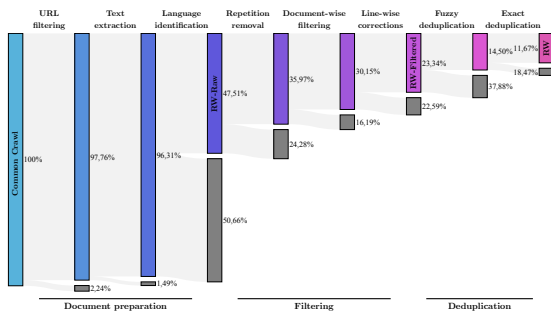
The full preprocessing processing pipeline (Falcon - RefinedWeb)



📄 Falcon Refined Web <https://huggingface.co/datasets/tiiuae/falcon-refinedweb>

Preparing Pretraining Data : Design Choices

Design choices



- Selecting sources: Web data vs. clean corpora (books, newspapers, specialized archives) vs. knowledge sources
- Balancing languages: how to counter the over-representation of English?
- Balancing genre, domains and text types (monolingual vs. parallel)
- Filtering target: code, non-language, hate, pornography, artificial text?
- Corpus quality evaluation

Improving Multi-Task Abilities with Instruction Tuning

Making LLMs better Multi-Task Performers

“Emerging” ability: zero-shot translation / few shot. From [Radford et al., 2019a]

We test whether GPT-2 has begun to learn how to translate from one language to another. In order to help it infer that this is the desired task, we condition the language model on a context of example pairs of the format `english sentence = french sentence` and then after a final prompt of `english sentence =` we sample from the model with greedy decoding and use the first generated sentence as the translation. (...)

So many undocumented issues

- over generation, under generation
- generation in the wrong language (out-of-target generation)
- translation errors and hallucinations: when **fluidity** does not entail **adequacy**

Improving Multi-Task Abilities with Instruction Tuning

Making LLMs better Multi-Task Performers

Improving translation ability

- 1 **Task Verbalization** turns exemplar translations into running text

This black dog is a wild beast. ||| Ce chien noir est une bête sauvage.

becomes:

Translate English: "This black dog is a wild beast."

into French: "Ce chien noir est une bête sauvage."

- 2 **Resume LLM training** with task-oriented texts and cross-entropy loss

Improving Multi-Task Abilities with Instruction Tuning

Making LLMs better Multi-Task Performers

Improving translation ability

- 1 **Task Verbalization** turns exemplar translations into running text

This black dog is a wild beast. ||| Ce chien noir est une bête sauvage.

becomes:

Translate English: "This black dog is a wild beast."

into French: "Ce chien noir est une bête sauvage."

- 2 **Resume LLM training** with task-oriented texts and cross-entropy loss

Design choices

- instructions formulation using **task templates**

Translate English: <English_text> into French: <French_text>

- tasks selection and balancing
- output format
- training regime and losses (full training vs parameter efficient fine tuning (PEFT))

Better Multi-Task Abilities with In-Context Learning

Making LLMs better Multi-Task Performers

Improving translation ability with in-context examples

- 1 **Task Verbalization**: turn exemplar translations into running text

This black dog is wild. ||| Ce chien noir est sauvage.

becomes:

[English:] "This black dog is wild." [French:] "Ce chien noir est sauvage."

- 2 **Inject example** in the prompt and continue generating

[English:] "This black dog is wild." [French:] "Ce chien noir est sauvage."

[English:] "This crazy cat drives me mad." [French:] ...

Better Multi-Task Abilities with In-Context Learning

Making LLMs better Multi-Task Performers

Improving translation ability with in-context examples

- 1 **Task Verbalization**: turn exemplar translations into running text

This black dog is wild. ||| Ce chien noir est sauvage.

becomes:

[English:] "This black dog is wild." [French:] "Ce chien noir est sauvage."

- 2 **Inject example** in the prompt and continue generating

[English:] "This black dog is wild." [French:] "Ce chien noir est sauvage."

[English:] "This crazy cat drives me mad." [French:] ...

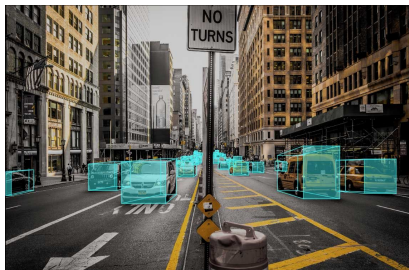
Design choices

- task template
- number of "shots" (examples)
- selection of "shots":
random vs. relevance based vs. quality based vs. diversity based
- presentation / ordering of shots

From Pre-Training to Tasks Accomplishment: Annotations Needed !

Annotations Fuel Fine-Tuning and In Context Learning

- pre-training learns from “natural” annotations (e.g., NWP)
- fine-tuning learns from “artificial”, task specific, annotations



Annotations require human labor and expertise

From Pre-Training to Tasks Accomplishment: Annotations Needed !

Text Classification

Sentence(s)	Yes / No	Detection of Spam, Hate Speech, Fake News, Toxic / Inappropriate Content
Sentence(s)	+ / - / =	Sentiment / Emotion Analysis, Stance Detection

Sequence Labelling

Text	Span+Label	Part-of-speech Tagging, Named Entity Recognition, Error Detection
------	------------	---

Answer Generation

Questions	A / B / C / D	Multi choice question answering
Text+Questions	Answer	Natural language Understanding
Question	Answer	General, Open Domain Question Answering

Text Generation

Source sentence(s)	Target	Translation, Automatic Post-edition
Sentence(s)	Summary	Text summarization, and Simplification
Sentences(s)	Corrections	Orthographic and Grammatical Correction, text normalization

Annotated Data Types for Instruction Tuning

 <https://huggingface.co>

Towards Prompt Engineering

Persona / Role	the human role that the system is asked to mimic
Directive	the core intent of the prompt, expressed as an instruction or a question
Examples	few-shot examples for in context learning
Input / Query	the text of the question, query or translation, etc
Formating	specify structural format requirements (eg., csv or json file), typography and disposition
Style	stylistic format requirements
Misc	additional directives (eg., regarding CoT)
Output	the expected system output (for fine tuning)

Complex prompts can also be used with instruction tuning

A excellent reference on prompting: Schulhoff et al. [2025]

Towards Prompt Engineering

A zero-shot prompt for hateful speech detection (lm-harness)

Is the following statement hateful? Respond with either Yes or No. Statement: [INPUT]

[ANSWER]

A excellent reference on prompting: Schulhoff et al. [2025]

Towards Prompt Engineering

A zero-shot prompt for Named Entity recognition (from Tower-Instruct)

Highlight all the named entities in the following list of tokens "[LIST]".

These are the annotation guidelines for named entities you need to follow:

- Person - Names of people
- Location - Location or physical facilities
- Group - Groups of people, organizations, corporations or other entities
- Product - Consumer products such as food, drinks, clothing, and vehicles
- CreativeWorks - Titles of creative works like movie, song, and book titles
- Medical - Entities from the medical domain, including diseases, symptoms, and medications.

Besides that, prepend B- as a prefix of the first token of any entity and I- to subsequent ones if they exist. If a token is not a part of a named entity, mark it as O.

Answer: [ANSWER]

LIST = ["Marcel", "Marcel", "a", "écrit", "Albertine", "disparue"]

ANSWER = [("Marcel", B-Person), ("Proust", I-Person), ("a", O), ("écrit", O), ("Albertine", B-CreativeWork), ("disparue", I-CreativeWork)]

A excellent reference on prompting: Schulhoff et al. [2025]

Towards Prompt Engineering

A 3-shot prompt for automatic translation correction (from Tower-Instruct)

Here are some examples of Spanish source-reference pairs. The source may contain errors, in which case the reference will be its corrected version. Otherwise, the reference will be a copy of the source.

Source: Su riza era contagiosa

Reference: Su risa era contagiosa

Source: Pronto, Brad Pitt estaba actuando en muchos peliculas

Reference: Más tarde, Brad Pitt fue descubierto

Source: Él es más famoso por las películas y episodios de Star Trek

Reference: Él es más famoso por las películas y episodios de Star Trek

Correct the errors in the following Spanish source, if there are any; if not, simply copy the sentence.

Source: [INPUT]

Reference: [OUTPUT]

A excellent reference on prompting: Schulhoff et al. [2025]

Towards Prompt Engineering

2-shot prompt for MMLU (algebra section), with chain-of-thoughts (from lm-eval)

The following are multiple choice questions (with answers) about abstract algebra.

Question: Statement 1 | Every element of a group generates a cyclic subgroup of the group.

Statement 2 | The symmetric group S_{10} has 10 elements.

(A) True, True (B) False, False (C) True, False (D) False, True. Let's think step by step.

Answer: A cyclic group is a group that is generated by a single element. Hence a subgroup generated by a single element of a group is cyclic and Statement 1 is True. The symmetric group S_n has $n!$ elements, hence it is not true that S_{10} has 10 elements. The answer is (C).

Question: Statement 1 | Every function from a finite set onto itself must be one to one. Statement 2 | Every subgroup of an Abelian group is Abelian.

(A) True, True (B) False, False (C) True, False (D) False, True. Let's think step by step.

Answer: Statement 1 is true. Let S be a finite set. If $f : S \rightarrow S$ is a onto function, then $|S| = |f(S)|$. If f was not one to one, then for finite domain S the image would have less than S elements, a contradiction. Statement 2 is true. Let G be an abelian group and H be a subgroup of G . We need to show that H is abelian. Let $a, b \in H$. Then $a, b \in G$ and $ab = ba$. Since G is Abelian, $ab = ba$. Since H is a subgroup of G , $ab \in H$. Therefore, $ab = ba$ and H is Abelian. The answer is (A).

Question: [QUESTION] Let's think step by step.

Answer: [OUTPUT]

A excellent reference on prompting: Schulhoff et al. [2025]

Towards Prompt Engineering

- The **design / formulation** of prompts matters - especially in zero-shot settings.
- The **language** of prompt matters (in multilingual settings)
- Prompt may be subject to **automatic revision, optimization and learning**
- Prompts may be **ensembled** to mitigate output variability and improve robustness
- Prompts may be subject to **adversarial attacks**

- The design of **answer extraction** also matters much

Prompts are just conditioning vectors - effects of prompt design are hard to anticipate

A excellent reference on prompting: Schulhoff et al. [2025]

Alignment with Human Values through RLHF

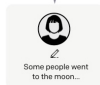
Step 1

Collect demonstration data, and train a supervised policy.

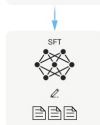
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

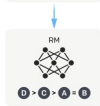
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



📖 [Ouyang et al., 2022] <https://openai.com/index/instruction-following/>

Alignment with Human Values through RLHF

Step 1

Collect demonstration data, and train a supervised policy.

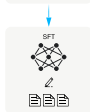
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Use-case	(%)	Examples
Classification	3.5%	Take the following text and rate, on a scale from 1-10, how sarcastic the person is being (1 = not at all, 10 = extremely sarcastic). Also give an explanation¶ {text}¶ Rating:
Closed QA	2.6%	Answer the following question: What shape is the earth? ¶ A: A circle B: A sphere C: An ellipse D: A plane
Open QA	12.4%	Who built the statue of liberty?
Generation	45.6%	Write a short story where a brown bear to the beach, makes friends with a seal, and then return home.
Rewrite	6.6%	Translate this sentence to Spanish: ¶ <English sentence>
Summarization	4.2%	Summarize this for a second-grade student: ¶ {text}
Extract	1.9%	Extract all place names from the article below: ¶ {news article}
Brainstorming	11.2%	List five ideas for how to regain enthusiasm for my career
Chat	8.4%	Marv is a chatbot that reluctantly answers questions with sarcastic responses:¶You: How many pounds are in a kilogram?¶Marv: This again? There are 2.2 pounds in a kilogram. Please make a note !s
Other	3.5%	

Alignment with Human Values through RLHF

Step (2) collect human preferences and learns a reward model

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.

The reward model is an auxiliary model evaluating (query, answers) pairs with a numerical value

- 1 Sample prompts $\{x_1 \dots x_n\}$, generate k automatic answers $\{(y_{i1}, \dots y_{ik})\}_{i=[1:n]}$
- 2 Ask humans to rank them $\{(r_{i1}, \dots r_{ik})\}_{i=[1:n]}, r_{ij} \in [1:k]$
- 3 train model $(x, y) \rightarrow r_{\lambda}(x, y)$ such that preferred answers get larger rewards. This is a **learning-to-rank setting**.

$$\text{loss}(\lambda) = -\frac{1}{\binom{k}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)))]$$

where y_w is a better answer than y_l on input x

Alignment with Human Values through RLHF

Repeat:

- ① Sample prompt x
- ② generate model answers (y) with current parameters θ
 $((x, y) \sim D_{\pi_{\theta}^{\text{RL}}})$
- ③ compute their reward $r_{\lambda}(x, y)$
- ④ update model parameters to prefer answers with higher rewards with a reinforcement learning loss (Proximal Policy Optimisation: PPO).

Step 3

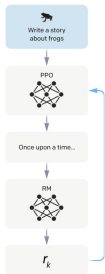
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



$$\ell_{\text{PPO}}(\phi) = E_{(x,y) \sim D_{\pi_{\theta}^{\text{RL}}}} \left[r_{\lambda}(x, y) - \beta \log \left(\frac{\pi_{\theta}^{\text{RL}}(y | x)}{\pi_{\theta_0}^{\text{SFT}}(y | x)} \right) \right] + \gamma E_{x \sim D_{\text{pretrain}}} [\log(\pi_{\theta}^{\text{RL}}(x))]$$

The loss contains two terms: the first is the RL objective, the second is the standard pretraining term, ensures that the model does not lose its “instruction”.

Alignment with Human Values through RLHF

TIME

SIGN UP FOR OUR IDEAS NEWSLETTER POV

SUBSCRIBE



BUSINESS • TECHNOLOGY

Exclusive: OpenAI Used Kenyan Workers on Less Than \$2 Per Hour to Make ChatGPT Less Toxic

15 MINUTE READ



This image was generated by OpenAI's image-generation software, Dall-E 2. The prompt was: "A seemingly endless view of African workers at desks in front of computer screens in a printmaking style." TIME does not typically use AI-generated art to illustrate its stories, but chose to in this instance in order to draw attention to the power of OpenAI's technology and shed light on the labor that makes it possible. Image generated by Dall-E 2/OpenAI

Alignment also requires human labor and expertise.

 <https://time.com/6247678/openai-chatgpt-kenya-workers/>

Generating with Language Models

Language Models are Distributions

Given **finite vocabulary** $\mathcal{V}$, with $\bar{\mathcal{V}} = \mathcal{V} \cup \{ \langle s \rangle, \langle /s \rangle \}$

A language model is a **parameterized distribution** over **complete texts** in $\langle s \rangle \mathcal{V}^* \langle /s \rangle$:

$$\begin{aligned} \langle s \rangle w_1 \dots w_T \langle /s \rangle &\rightarrow P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) \\ \forall T > 0, \forall w_1 \dots w_T, P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) &\geq 0, \\ \sum_{T, w_{[1:T]}} P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) &= 1 \end{aligned}$$

Main assumption: **high-probability texts** $\Leftrightarrow$ **high-quality texts**

Generating with Language Models

Language Models are Distributions

Given **finite vocabulary** $\mathcal{V}$, with $\bar{\mathcal{V}} = \mathcal{V} \cup \{ \langle s \rangle, \langle /s \rangle \}$

A language model is a **parameterized distribution** over **complete texts** in $\langle s \rangle \mathcal{V}^* \langle /s \rangle$:

$$\begin{aligned} \langle s \rangle w_1 \dots w_T \langle /s \rangle &\rightarrow P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) \\ \forall T > 0, \forall w_1 \dots w_T, P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) &\geq 0, \\ \sum_{T, w_{[1:T]}} P(\langle s \rangle w_1 \dots w_T \langle /s \rangle \mid \theta) &= 1 \end{aligned}$$

Main assumption: **high-probability texts** $\Leftrightarrow$ **high-quality texts**

Formalizing Text Generation as Search

Unconditional Text Generation: find “most probable text”

$$[w_1^* \dots w_{T^*}^*] = \operatorname{argmax}_{T, [w_{1:T}]} P([w_{1:T}] | \theta)$$

Finding T^* is part of the problem. Exact search is **untractable**.

Conditional Text Generation: find “most probable response” given input query (MAP)

$$[w_1^* \dots w_{T^*}^*] = \operatorname{argmax}_{T, [w_{1:T}]} P([w_{1:T}] | \mathbf{C}, \theta)$$

$\mathbf{C}$: a textual prefix/prompt (**text completion**), a question (**question answering**), a source text (**translation**), a long text (**summarization**), demonstrations (**in context learning**, retrieved documents (RAG), a speech file (**transcription**), an image (**captioning**), ...)

A variety of situations between **open set generation** (many acceptable texts) and **near deterministic generation** (one single acceptable output).

Formalizing Text Generation as Search

Looking for the Mode (I)

$$[w_1^* \dots w_{T^*}^*] = \operatorname{argmax}_{T, [w_{1:T}]} P([w_{1:T}] | \theta)$$

$$\text{also: } \operatorname{argmax}_{T, [w_{1:T}]} P([w_{1:T}] | \mathbf{C}; \theta)$$

$$= \operatorname{argmax}_{T, [w_{1:T}]} \prod_{t=1}^{T+1} P(w_t | w_{<t}; \theta)$$

Chain rule for **autoregressive / causal** LMs

$$= \operatorname{argmax}_{T, [w_{1:T}]} \log \prod_{t=1}^{T+1} P(w_t | w_{<t}; \theta)$$

log is monotonous

$$= \operatorname{argmin}_{T, [w_{1:T}]} \sum_{t=1}^{T+1} -\log P(w_t | w_{<t}; \theta)$$

log turns $\prod$ into $\sum$

- $-\log P(w_t | w_{<t}; \theta) > 0$ is the **surprisal**; upper bounded by $\log |\mathcal{V}|$
quantifies how much w_t was expected given $w_{<t}$, used in **psycholinguistic studies**
- $\max_{T, [w_{1:T}]} P([w_{1:T}] | \theta)$ equivalently minimizes a summation of $T + 1$ surprisals

Formalizing Text Generation as Search

Looking for the Mode (II)

$$P(w | w_{<t}; \theta) = \frac{\exp \text{logit}(w, w_{<t}; \theta)}{\sum_{w' \in \mathcal{V}} \exp \text{logit}(w', w_{<t}; \theta)}$$

$$\log P(w | w_{<t}; \theta) = \text{logit}(w, w_{<t}; \theta) - \log \sum_{w' \in \mathcal{V}} \exp \text{logit}(w', w_{<t}; \theta)$$

$$\begin{aligned} [w_1^* \dots w_{T^*}^*] &= \underset{T, [w_{1:T}]}{\operatorname{argmin}} \left(\sum_{t=1}^{T+1} \text{logit}(w_t, w_{<t}; \theta) - \log \sum_{w'} \exp \text{logit}(w', w_{<t}; \theta) \right) \\ &= \underset{T, [w_{1:T}]}{\operatorname{argmin}} \sum_{t=1}^{T+1} \text{logit}(w_t, w_{<t}; \theta) \end{aligned}$$

- X a finite set, $f : X \rightarrow \mathbb{R}$ a real function, $\frac{\exp f(x)}{\sum_{x' \in X} \exp f(x')}$ is the **softmax**
- $\text{softmax}(x)$ is always > 0 ; almost 1 for the largest $f(x)$, almost 0 otherwise
- computing the logits requires a **full forward pass** in Transformers ($O(T \times L \times d_{\text{model}})$)
- **normalizer** $\sum_{w' \in \mathcal{V}} \exp \text{logit}(w', w_{<t}; \theta)$ can be expensive to compute ($\sum$ over $|\mathcal{V}|$ terms)

Algorithms for Text Generation

Deterministic (MAP style) Generation with Greedy search

$$w_0 = \textcolor{blue}{<s>}$$

$$w_t = \operatorname{argmax}_{w \in \bar{\mathcal{V}}} \log P(w \mid w_{<t}; \theta) \quad (\text{for } t > 0)$$

$\bar{\mathcal{V}} = \mathcal{V} \cup \{\textcolor{blue}{<s>}, \textcolor{blue}{</s>}\}$. Generation stops with the $\textcolor{blue}{</s>}$ symbol or when some maximum time step T is reached. Greedy search is a simple way to approximate the untractable $\operatorname{argmax}_{w_{[1:T]}} P(w_{[1:T]})$. A better way: [beam-search](#).

Algorithms for Text Generation

Non Deterministic Generation: Ancestral sampling

$$w_0 = \text{<s>}$$

$$w_t \sim P(w | w_{<t}; \theta) \quad (\text{for } t > 0)$$

Generation stops with the `</s>` symbol or when some maximum time step T is reached.

- Sampling is **non-deterministic**: output varies, depending on the sharpness of $P(w | w_{<t}; \theta)$
- softmax is very peaked: increase diversity with **temperature** τ :

$$P(w | w_{<t}; \theta) \propto \exp \frac{\text{logit}(w, w_{<t}; \theta)}{\tau}$$

$\tau > 1$ has the effect of “flattening” the output distribution

Algorithms for Text Generation

Top-k sampling

$$w_0 = \textcolor{blue}{<s>}$$

$$Q(w_t | w_{<t}; \theta) \propto \begin{cases} P(w_t | w_{<t}; \theta) & \text{if } w \in \text{top-}k(P(W | w_{<t}; \theta)) \\ 0 & \text{otherwise} \end{cases}$$

$$w_t \sim Q(w | w_{<t}; \theta) \quad (\text{for } t > 0)$$

Sample from a **“truncated” distribution** containing the k most likely symbols. Generation stops with the $\textcolor{blue}{</s>}$ symbol or when some maximum time step T is reached.

Algorithms for Text Generation

Nucleus sampling (top p , with variable p)

$$w_0 = \text{<s>}$$

$$Q(w_t | w_{<t}; \theta) \propto \begin{cases} P(w_t | w_{<t}; \theta) & \text{if } w \in \text{top-}p(P(W | w_{<t}; \theta)) \\ 0 & \text{otherwise} \end{cases}$$

$$w_t \sim Q(w | w_{<t}; \theta) \quad (\text{for } t > 0)$$

Where p is the smallest integer such that $\sum_{w \in \text{top-}p} P(w | w_{<t}; \theta) > \alpha$. Sample from a **“truncated” distribution** for the p most likely symbols, with variable p (α typically $\in [0.7; 0.9]$). Generation stops with the **</s>** symbol or when some maximum time step T is reached.

Algorithms for Text Generation

And many more:

- local sampling from **proxy distributions** $\sim q_\theta$
e.g. temperature sampling, η , ϵ -sampling, with repetition penalty, adaptive sampling
- advanced search techniques and **global constraints** (e.g., MCTS)
- multi-step procedures:
 - n -best generation + ex-post selection
(e.g, **Minimum Bayes Risk decoding**)
 - hypothesis recombination (e.g., lattice decoding)
 - iterative refinement
 - “chain-of-thoughts”

See [Wiher et al., 2022, Welleck et al., 2024]

Language Model (De)Generation

The promise

🔗 OpenAi Website <https://openai.com/blog/better-language-models/>

GPT-2 generates synthetic text samples in response to the model being primed with an arbitrary input. The model is chameleon-like—it adapts to the style and content of the conditioning text. This allows the user to generate **realistic and coherent continuations** about a topic of their choosing, as seen by the following select samples.

GPT-2 displays a broad set of capabilities, including the ability to **generate conditional synthetic text samples of unprecedented quality**, where we prime the model with an input and have it generate a lengthy continuation.

Language Model (De)Generation

The truth about language model generation with GPT2

Prefix	Lyrically the song has excerpts of different languages including French, Spanish,
Greedy	German, Italian, Portuguese, Spanish, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese, Portuguese , ...
top-k [3]	German, Italian, Portuguese, Spanish, Portuguese, Portuguese, Italian and Spanish. It is also available in English, French and Spanish. In addition to its lyrics, the album features a number of original songs, ...
top-p [0.3]	German, Italian, Portuguese, Spanish, Portuguese, Italian, Portuguese, Spanish, Portuguese, Spanish, Portuguese, Spanish, Portuguese, Spanish, Portuguese, Spanish, Portuguese, ...
Prefix	The first pair of pereopods is armed with a large, asymm
Greedy	etrical, and long-range laser cannon. The second pair is armed with a large, asymmetrical, and long-range laser cannon. The third pair is armed with a large, asymmetrical, and long-range laser cannon. The fourth pair is armed with a large ...
top-k [3]	etrical, and highly mobile head, and the second pair has a small, asymmetrical, and highly mobile head. The second pair has a large and highly mobile head, and the third pair is armed with a large and highly mobile head. The first pair ...
top-p [0.3]	etrical head and a large body. The first pair of pereopods is armed with a large, asymmetrical head and a large body. The first pair of pereopods is armed with a large, asymmetrical head and a large body. The first pair of pereopods is armed ...

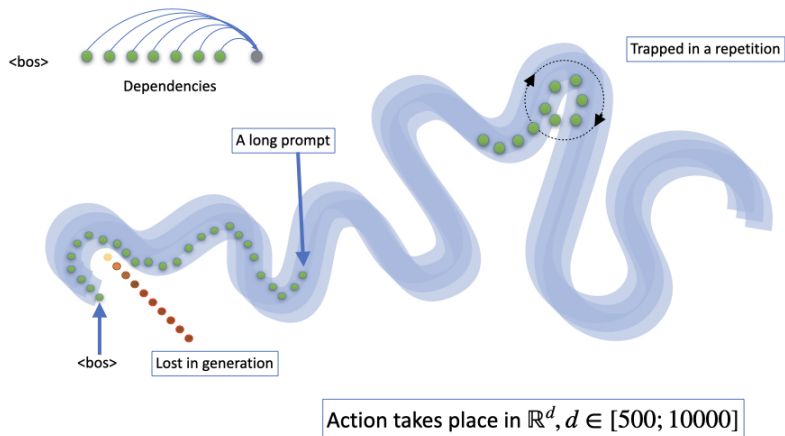
Examples from [Welleck et al., 2019].

Language Model (De)Generation

High probability sentences do not resemble human productions

- too many repetitions
- high frequency tokens over-represented, low frequency tokens under-represented
- lack of **lexical diversity**
- lack of **global consistency**
- **hallucinations** all over the place

Language Model (De)Generation



Part III

Using Large language Models

Evaluating LMs, a very serious matter

In recent years, the AI technology that has arguably advanced the most is foundation models (Bommasani et al., 2021), headlined by the rise of language models (LMs; Peters et al., 2018; Devlin et al., 2019; Brown et al., 2020; Rae et al., 2021; Chowdhery et al., 2022). At its core, a language model is a box that takes in text and generates text (...) Yet the immense surface of model capabilities, limitations, and risks remains poorly understood. The rapid development, rising impact, and inadequate understanding demand that we benchmark language models holistically. [Liang et al., 2022]

The multiple dimensions of LM evaluation

As text generators	As representation learners
Good models of actual texts?	Recovering human-like associations?
Discriminating well-formed utterances?	Discovering linguistic relationships?
Learning linguistic structures	Useful for downstream applications?
Generating realistic texts?	- in 'unstructured' learners?
Enabling controlled generation?	- in 'structured' learners?

Evaluating LMs, a very serious matter

In recent years, the AI technology that has arguably advanced the most is foundation models (Bommasani et al., 2021), headlined by the rise of language models (LMs; Peters et al., 2018; Devlin et al., 2019; Brown et al., 2020; Rae et al., 2021; Chowdhery et al., 2022). At its core, a language model is a box that takes in text and generates text (...) Yet the immense surface of model capabilities, limitations, and risks remains poorly understood. The rapid development, rising impact, and inadequate understanding demand that we benchmark language models holistically. [Liang et al., 2022]

The multiple dimensions of LM evaluation

As text generators	As representation learners
Good models of actual texts?	Recovering human-like associations?
Discriminating well-formed utterances?	Discovering linguistic relationships?
Learning linguistic structures	Useful for downstream applications?
Generating realistic texts?	- in 'unstructured' learners?
Enabling controlled generation?	- in 'structured' learners ?

Evaluating Language Models with Perplexity

Perplexity of a test sequence $w_{[1:T]}$ [Brown et al., 1992]

$$\text{PPL}(M) = 2^{\frac{-1}{T} \log_2 P(w_{[1:T]} | M)} = P(w_{[1:T]} | M)^{-\frac{1}{T}}$$

- The **cross-entropy** between the source (S) and model M (in bits):

$$H(S, M) = \lim_{T \rightarrow \infty} -\frac{1}{T} \log_2 P(w_{[1:T]} | M)$$

$H(S, M)$ upper bounds $H(S)$

- $\text{PPL}()$ **homogeneous to a vocabulary size**

$$\text{PPL}(\text{Unif}) = 2^{\frac{-1}{T} \log_2 P(w_{[1:T]} | M)} = 2^{\frac{-1}{T} T \log_2 (1/n_W)} = n_W$$

Evaluating Language Models with Perplexity

Comparing LMs with different support or segmentations?

- ① closed-world LMs assume a fixed vocabulary size n_W - models with different support **cannot be compared**.
- ② open-world models with different **segmentations can be compared**
 - using the same texts ($\frac{1}{T}$)
 - and the same normalizer
- ③ typical normalizers for subwords based models
 - number of chars $T_c \Rightarrow$ **bits per char** : $-\frac{1}{T_c} \log_2 P(w_{[1:T]} | M)$
 - number of bytes $T_b \Rightarrow$ **bits per byte** : $-\frac{1}{T_b} \log_2 P(w_{[1:T]} | M)$

Evaluating zero-shot / few-shot behaviour

Reduce NLP tasks to text generation with appropriate NL instructions as prompts

Prompts = instructions in Natural Language + [tricks] (from [Brown et al., 2020])

Specifically, we evaluate GPT-3 on over two dozen NLP datasets,(...) For each task, we evaluate GPT-3 under 3 conditions:

- “zero-shot” learning, where no demonstrations are allowed and only an instruction in natural language is given to the model.

“Evaluate $125 + 12 =$ ”

- “one-shot learning”, where we allow only one demonstration, and

“Evaluate $17 + 301 = 318$ </s> Evaluate $125 + 12 =$ ”

- “few-shot learning”, or “in-context learning”, where we allow as many demonstrations as will fit into the model’s context window,

“Evaluate $17 + 301 = 318$ </s> Evaluate $48 + 67 = 105$ </s> Evaluate $125 + 12 =$ ”

Tricks: “On tasks with free-form completion, we use beam search with the same parameters as [RSR+19]: a beam width of 4 and a length penalty of $\alpha = 0.6$.” (+ stopping criterion)

Evaluating zero-shot / few-shot behaviour

Reduce NLP tasks to text generation with appropriate NL instructions as prompts

Task types and their evaluation

Assuming prompt / instruction: $w_1 \dots w_T$.

- Yes / No answers

Question: *[Question]* True or false? *[prediction]*

Correct if $P(\text{True} | \text{prompt}) > P(\text{False} | \text{prompt})$.

- Multiple choice answers.

Question: *Which factor will most likely cause a person to develop a fever?*

Correct Answer *a bacterial population in the bloodstream*

Incorrect Answer *a leg muscle relaxing after exercise*

Incorrect Answer *several viral particles on the skin*

Incorrect Answer *carbohydrates being digested in the stomach*

Correct if $P(\text{Correct answer} | \text{prompt}) > P(\text{Alternative} | \text{prompt})$

- One word continuation. Correct if $w_{T+1} == w^*$
- Multiple word continuation. Measure $\Delta(w_{T+1} \dots w_{T+S}; w_1^* \dots w_L^*)$ with $\Delta()$ task-dependent distance (ROUGE for summarization, BLEU for MT, etc)

Evaluating zero-shot / few-shot behaviour

Reduce NLP tasks to text generation with appropriate NL instructions as prompts

Understanding “instruction learning” results

Should pay attention to:

- how much effort went into prompting?
- free generation or text infilling or multi-choice answers?
- how were alternatives selected / generated?
- how was search performed (greedy or beam)?
- how does generation stops?
- how many shots is few shots?

Evaluating zero-shot / few-shot behaviour

Reduce NLP tasks to text generation with appropriate NL instructions as prompts

Understanding “instruction learning” results

Should pay attention to:

- how much effort went into prompting?
- free generation or text infilling or multi-choice answers?
- how were alternatives selected / generated?
- how was search performed (greedy or beam)?
- how does generation stops?
- how many shots is few shots?

Open / new issues

- generating / optimizing discrete / continuous prompts
- optimizing retrieval and demonstration examples
- training with prompts and meta-learning

LLMs in HCI Research

Where have LLMs been applied in CHI papers?

Communication & Writing	On various writing and communication tasks, which often target writers as the primary user groups.
Augmenting Capabilities	On technologies to enhance human performance and productivity, often in the physical world.
Education	On learning experiences for students and pedagogical methods for educators.
Responsible Computing	On ethical and societal implications of computational systems, particularly in high-stakes domains.
Programming	On various aspects of software development and programming tasks.
Reliability & Validity of LLMs	On evaluating and improving LLM outputs themselves.
Well-being & Health	On managing health-related disorders/illnesses, or interactions with health data or healthcare providers.
Design	On various types of design work, which often target designers as the primary user group.
Accessibility & Aging	On population with disabilities and older adults.
Creativity	On the creativity process and creativity support tools, which often overlaps with other domains.

📖 Yuren et al, 2025 <https://arxiv.org/abs/2501.12557v1>

LLMs in HCI Research

How do HCI papers leverage these models?

LLMs as system engines	LLMs function as core elements within systems, prototypes, algorithms, and programming frameworks.
LLMs as research tools	LLMs perform research tasks traditionally executed by researchers in a research project, such as data collection, analysis, and writing.
LLMs as participants & users	LLMs simulate human responses and behaviors, or act as users or participants in an interaction.
LLMs as objects of study	LLMs' inner mechanism, properties, performance are evaluated.
Users' perceptions of LLMs	LLMs or tools (e.g., ChatGPT) are studied to understand user perceptions in different contexts.

📖 Yuren et al, 2025 <https://arxiv.org/abs/2501.12557v1>

Limitations, Barriers and Risks

Limitations

Lack of:

- Explainability
- Transparency
- Robustness
- Systematicity
- Calibration
- Factuality
- Edition, incremental update
- Adaptation
- Personalization

Barriers

Linguistic Weaknesses

- Multilingualism
- Language dynamics
- Language variability
- Linguistic creativity

High-level dis-abilities

- Reasoning and Planning
- Semantic grounding
- World Knowledge

Risks

Text Generation

- Biases & stereotypes
- Hallucinations / confabulations
- Toxicity, Hate and Fakes
- Privacy breach

Data issues

- Data theft and plundering
- Data pollution
- Data & Model poisoning

Computation

- Consumption and Carbon Emissions
- “Monopolization” of HPC

Limitations, Barriers and Risks

Limitations

Lack of:

- Explainability
- Transparency
- Robustness
- Systematicity
- Calibration
- Factuality
- Edition, incremental update
- Adaptation
- Personnalization

Barriers

Linguistic Weaknesses

- Multilingualism
- Language dynamics
- Language variability
- Linguistic creativity

High-level dis-abilities

- Reasoning and Planning
- Semantic grounding
- World Knowledge

Risks

Text Generation

- Biases & stereotypes
- Hallunications / confabulations
- Toxicity, Hate and Fakes
- Privacy breach

Data issues

- Data theft and plundering
- Data pollution
- Data & Model poisoning

Computation

- Consumption and Carbon Emissions
- “Monopolization” of HPC

Limitations, Barriers and Risks

Limitations

Lack of:

- Explainability
- Transparency
- Robustness
- Systematicity
- Calibration
- Factuality
- Edition, incremental update
- Adaptation
- Personnalization

Barriers

Linguistic Weaknesses

- Multilingualism
- Language dynamics
- Language variability
- Linguistic creativity

High-level dis-abilities

- Reasoning and Planning
- Semantic grounding
- World Knowledge

Risks

Text Generation

- Biases & stereotypes
- Halluncinations / confabulations
- Toxicity, Hate and Fakes
- Privacy breach

Data issues

- Data theft and plundering
- Data pollution
- Data & Model poisoning

Computation

- Consumption and Carbon Emissions
- “Monopolization” of HPC

Limitations, Barriers and Risks

Category	Source	Correct Translation	Hallucinatory Translation
Intrinsic	迈克周四去书店。	Mike goes to the bookstore on Thursday.	Jerry doesn't go to the bookstore on Thursday.
Extrinsic	迈克周四去书店。	Mike goes to the bookstore on Thursday.	Mike happily goes to the bookstore on Thursday with his friend.
Detached	Das kann man nur feststellen, wenn die kontrollen mit einer großen intensität durchgeführt werden.	This can only be detected if controls undertaken are more rigorous.	Blood alone moves the wheel of history, i say to you and you will understand, it is a privilege to fight.
Oscillatory	1995 das produktionsvolumen von 30 millionen pizzen wird erreicht.	1995 the production reached 30 million pizzas.	The US, for example, has been in the past two decades, but has been in the same position as the US, and has been in the United States.

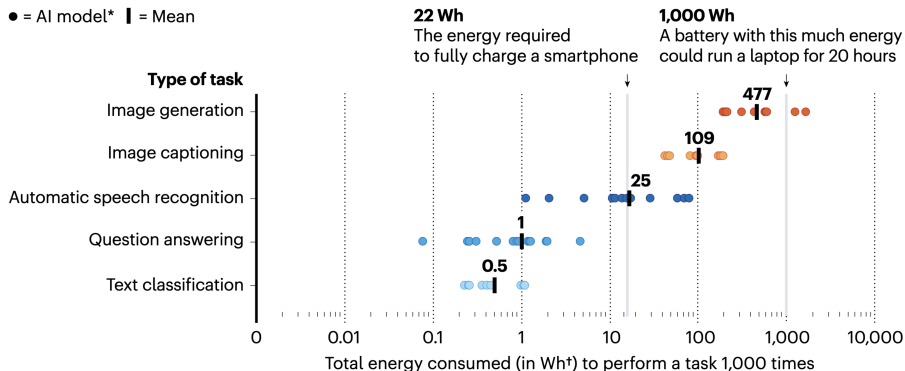
The hallucinations of Machine Translation

Limitations, Barriers and Risks

AI'S ENERGY FOOTPRINT

The power consumed by artificial intelligence (AI) tools varies greatly depending on the task. An AI model that provides answers to queries is much less energy-intensive than one that generates images from text prompts, for example. And the data show that even AI models of the same type can vary widely in energy consumption.

● = AI model* | = Mean



*Tests conducted on 20 popular open-source models. Each dot represents one model; †1 Watt-hour represents power consumption of 1 W extended over 1 hour.

Source: Luccioni et al, Nature 632, 736-738 (2024)

Using LLMs in the Transformers Library

basic text generation

```
huggingface-cli login
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
tokenizer = AutoTokenizer.from_pretrained("utter-project/EuroLLM-9B")
model = AutoModelForCausalLM.from_pretrained("utter-project/EuroLLM-9B")
```

```
model_inputs = tokenizer(["A sequence of numbers: 1, 2"], return_tensors="pt").to("auto")
```

```
generated_ids = model.generate(**model_inputs)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```

```
generated_ids = model.generate(**model_inputs, max_new_tokens=50)
tokenizer.batch_decode(generated_ids, skip_special_tokens=True)[0]
```



<https://huggingface.co/utter-project/EuroLLM-9B?library=transformers>

Implementations and models available online

- 👉 <https://transformer.huggingface.co/>
- 👉 <https://github.com/facebookresearch/fairseq>
- 👉 <https://www.tensorflow.org/hub?hl=en>
- 👉 <https://github.com/vllm-project/vllm>

Current challenges for language modeling

- 1 more than one language: multilingual LLMs
- 2 find the right modeling units (between char-based and word-based)
- 3 model very long range context-dependency beyond syntax: [Dai et al., 2019] repetitions, style, discourse consistency, etc.
- 4 towards better text generation through better searching - simulating reasoning
- 5 improve training efficiency / scalability; reduce the carbon emissions of large-scale models
- 6 (continuous) online adaptation of LMs - is Retrieval Augmented Generation the solution?
- 7 mitigate the biases of “stochastic parrots” (also: privacy issues, etc) [Bender et al., 2021]

Bibliography I

- Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, FAccT '21, pages 610–623, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383097. doi: 10.1145/3442188.3445922. URL <https://doi.org/10.1145/3442188.3445922>.
- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003. ISSN 1532-4435.
- Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra, Jennifer C. Lai, and Robert L. Mercer. An estimate of an upper bound for the entropy of English. *Computational Linguistics*, 18(1):31–40, 1992. URL <https://aclanthology.org/J92-1002>.

Bibliography II

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 2104–2114, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.161. URL <https://aclanthology.org/2021.emnlp-main.161>.

Bibliography III

- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *CoRR*, abs/1901.02860, 2019. URL <http://arxiv.org/abs/1901.02860>.
- Hai Son Le, Alexandre Allauzen, Guillaume Wisniewski, and François Yvon. Training continuous space language models: Some practical issues. In *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing*, pages 778–788, Cambridge, MA, 2010. URL <http://www.aclweb.org/anthology/D/D10/D10-1076>.
- Hai-Son Le, Alexandre Allauzen, and François Yvon. Continuous space translation models with neural networks. In *Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 39–48, Montréal, Canada, June 2012. Association for Computational Linguistics.

Bibliography IV

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models, 2022. URL <https://arxiv.org/abs/2211.09110>.

Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. Recurrent neural network based language model. In Takao Kobayashi, Keikichi Hirose, and Satoshi Nakamura, editors, *Eleventh Annual Conference of the International Speech Communication Association (InterSpeech)*, pages 1045–1048, Makuhari, Chiba, Japan, 2010. URL https://www.isca-speech.org/archive/archive_papers/interspeech_2010/i10_1045.pdf.

Bibliography V

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *Proceedings of the International Conference on Learning Representations, ICLR*, 2022. URL <https://arxiv.org/abs/2108.12409>.
- Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019a.

Bibliography VI

- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019b. URL <https://openai.com/blog/better-language-models/>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21: 1–67, 2020.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2025. URL <https://arxiv.org/abs/2406.06608>.
- Holger Schwenk. Continuous space language models. *Computer, Speech and Language*, 21(3):492–518, 2007. ISSN 0885-2308.

Bibliography VII

- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. URL <https://www.aclweb.org/anthology/P16-1162>.
- Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Comput. Surv.*, apr 2022. ISSN 0360-0300. doi: 10.1145/3530811. URL <https://doi.org/10.1145/3530811>. Just Accepted.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5998–6008. Curran Associates, Inc., 2017.
- Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*, 2019.

Bibliography VIII

- Sean Welleck, Amanda Bertsch, Matthew Finlayson, Hailey Schoelkopf, Alex Xie, Graham Neubig, Ilia Kulikov, and Zaid Harchaoui. From decoding to meta-generation: Inference-time algorithms for large language models, 2024. URL <https://arxiv.org/abs/2406.16838>.
- Gian Wiher, Clara Meister, and Ryan Cotterell. On decoding strategies for neural text generators. *Transactions of the Association for Computational Linguistics*, 10: 997–1012, 2022. doi: 10.1162/tacl_a_00502. URL <https://aclanthology.org/2022.tacl-1.58>. Place: Cambridge, MA Publisher: MIT Press.